Rockchip Android MediaCodec Extended Parameters Support Instruction

ID: RK-SM-YF-E20

Release Version: V1.0.1

Release Date: 2025-02-07

Security Level: □Secret □Internal □Public ■Public

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily introduces all vendor-specific MediaCodec extension parameters supported on the Rockchip Android platform, aiming to help developers understand the extended features supported by the platform's codecs, and utilize the extension parameters as needed in their own MediaCodec applications.

> This document introduces extended parameters based on MediaCodec Codec2 and is only compatible with Android 12 and above versions.

Chip Name	Kernel Version
Support all chipsets	Linux-4.19, Linux-5.10, Linux-6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical Support Engineer

Software Development Engineer

Revision History

Version	Author	Date	Change Description
V1.0.0	Chen Jinsen	2024-09-02	Initial version release
V1.0.1	Chen Jinsen	2025-02-07	Added some extended parameter configurations

---

Contents

---

1. Extended Parameter Support for Decoder Platform

1.1 Low Latency Decoding Mode

The platform's low-latency decoding mode is used for rapid output of decoded images to enhance the decoding real-time performance.

This mode enables the internal fast frame parsing functionality of the decoder and ignores the frame's reference sequence, immediately outputting the decoded image. It is typically applied in scenarios sensitive to the latency of the decoding and display, such as screen casting and live streaming.

> Low-Latency Decoding Mode is a "one-in-one-out" type, and is not applicable to bitstreams with B-frames (B-frames require references from both preceding and following frames, and immediate output would cause decoded output frames to be out of order).

Enabling Methods:

```
mediaFormat.setInteger("low-latency", 1);
```

Confirm Activation Log:

```
c2_info("enable lowLatency, enable mpp fast-out mode");
```

1.2 Tunneled Multimedia Tunnel Mode

Multimedia Tunneling Mode allows the decoder to establish a direct tunnel with the display driver, bypassing the Android SurfaceFlinger display framework pipeline. The decoded output is directly sent to the display without undergoing the SurfaceFlinger pipeline.

Application Scenario:

1. **High-bitrate, high-frame-rate video playback** - Bypassing the system's display framework pipeline reduces software overhead from system pipeline execution, releases system resources, to support high-bitrate, high-frame-rate video playback.
2. **Low-latency Display Solution** - The decoded output bypasses the SurfaceFlinger display framework and is directly synchronized to the display VOP, significantly reducing the display latency.

> Refer to Google's official website: Multimedia Tunneling Mode

1) Patch Configuration

Environment Requirements - Kernel version >= 5.10

1. Enable the rkvtunnel driver in your board-level DTS.

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
index 8c1d0f37b8a3..8bd65c9ed85b 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
@@ -846,6 +846,10 @@ &usbhost_dwc3_0 {
        status = "disabled";
 };
```

```
+&rkvtunnel {
+        status = "okay";
+};
+
 /* vp0 & vp3 are not used on this board */
 &vp0 {
        /delete-property/ rockchip,plane-mask;
```

2. In the decoder configuration file media_codecs.xml, declare that the decoder supports tunneling playback.

The following configuration file configures RK3588 AVC decoder to support tunnel mode. Similar configuration support applies for other chips and decoders.

```
~/3_android_14/vendor/rockchip/common/vpu/etc$ git diff .
diff --git a/vpu/etc/media_codecs_c2_rk3588.xml
b/vpu/etc/media_codecs_c2_rk3588.xml
index 28c1f947..d01da2d5 100644
--- a/vpu/etc/media_codecs_c2_rk3588.xml
+++ b/vpu/etc/media_codecs_c2_rk3588.xml
@@ -20,6 +20,7 @@
     </Settings>
     <Decoders>
         <MediaCodec name="c2.rk.avc.decoder" type="video/avc">
+            <Feature name="tunneled-playback" required="true"/>
             <Alias name="OMX.rk.video_decoder.avc" />
             <Limit name="size" min="64x64" max="7680x4320" />
             <Limit name="alignment" value="2x2" />
```

2) Application Notes

Multimedia Tunnel Mode audio-video synchronization methods support AUDIO_HW_SYNC/HW_AV_SYNC/REALTIME. The AUDIO_HW_SYNC and HW_AV_SYNC methods rely on the system tuner driver.

Currently supports only real-time rendering mode, i.e., latency-priority, where decoded output images are immediately retrieved and transmitted via tunnel for display. The program pseudocode is as follows:

```
// 1. get tunneled codec name
format.setFeatureEnabled(MediaCodecInfo.CodecCapabilities.FEATURE_TunneledPlayba
ck, true);
MediaCodecList mcl = new MediaCodecList(MediaCodecList.ALL_CODECS);
String codecName = mcl.findDecoderForFormat(format);

// 2. start tunneled video playback
MediaCodec mCodec = MediaCodec.createByCodecName(codecName);
mCodec.configure(format, mSurface, null, 0);
mCodec.start();

while (!isEOS) {
    int inIndex = mCodec.dequeueInputBuffer(0);
    if (inIndex >= 0) {
        ByteBuffer buffer = mCodec.getInputBuffer(inIndex);
        int sampleSize = mExtractor.readSampleData(buffer, 0);
        if (sampleSize > 0) {
            mCodec.queueInputBuffer(inIndex, 0, sampleSize,
```

```
                    mExtractor.getSampleTime(), 0);
            mExtractor.advance();
        } else {
            mCodec.queueInputBuffer(inIndex, 0, 0,
                    0, MediaCodec.BUFFER_FLAG_END_OF_STREAM);
            isEOS = true;
        }
    }
}

mCodec.stop();
mCodec.release();
```

1.3 Ignore H264 Bitstream Frame Sequence Continuity Check (Ignore Frame Drop Errors)

【Issue Description】

Real-time streaming scenarios such as network conferencing/monitoring may result in bitstream frame loss and packet loss due to network instability.

The decoder internally checks the continuity of the bitstream frame sequence by default. Upon detecting discontinuity in the POC of the bitstream, it marks the entire GOP sequence frames as error and discards all error frames within the GOP sequence. Display and playback only resume with the arrival of the next key frame.

Exception logs during the decoding process are as follows:

```
C2RKMpiDec: skip error frame with pts 0
```

【Problem Solution】

The decoder hardware has error correction function. Minor frame loss or packet loss in the bitstream can sometimes be recovered through the hardware's error correction functions. In such scenarios, the POC continuity check can be ignored and the bitstream can playback normally.

- Note that hardware error correction is not limitless, and excessively severe frame loss can still lead to color banding.
- POC continuity check is a standard process for bitstream decoding and is therefore supported solely as a user configuration.

In actual network stream transmission scenarios, if minor frame loss is detected causing image lagging, the following configuration can be used to disable the bitstream POC continuity check.

Enabling Methods:

```
mediaFormat.setInteger("vendor.c2-dec-disable-dpb-check.value", 1);
```

Confirmation of Effective Log:

```
c2_info("disable poc discontinuous check");
```

1.4 Disable Decoder Internal Error Handling

Regarding the compatibility of decoding abnormal bitstreams, such as reference relationship anomalies, bitstream syntax anomalies, etc.

Use the following user extension parameters to disable the internal error handling of the decoder. Once enabled, the decoder will ignore any error conditions in the bitstream and output all decodable images, while not marking the decoded output with errinfo.

> Note: Since the framework relies on the decoder's output of errinfo for frame-dropping processing, the absence of errinfo output may result in abnormal frames being displayed, causing screen artifacts.

Enabling Methods:

```
mediaFormat.setInteger("vendor.c2-dec-disable-error-mark.value", 1);
```

Confirmation of Effective Log:

```
c2_info("disable error frame mark");
```

1.5 Decoder Memory Usage Optimization Configuration

In scenarios where memory is constrained, such as on low-memory devices or other devices with limited available memory, user configurations are provided to bypass framework restrictions. This allows for reducing the number of output buffers requested by the decoder, thereby decreasing memory usage.

In the scenario of decoding and playback for high-resolution video sources, the memory optimization effects are particularly significant. Configuration methods:

```
mediaFormat.setInteger("vendor.c2-dec-low-memory-mode.value", xxx);

// Currently, two configuration options are supported:
// Configuration Option 0x1: Bypass framework restrictions and reduce the number
of buffer allocations by 4.
// Configuration Option 0x2: Actively detect and use the number of reference
frames in the bitstream SPS as the number of output buffer allocations.

// Set value 0x3 to configure and enable all optimization options.
```

Effective Log Confirmation:

```
c2_info("in low memory mode %d, reduce output ref count", lowMemoryMode);
```

> The decoder defaults to predicting the number of output buffers based on the bitstream level information, which allocates more memory and ensures compatibility with all sources. Configuration option 0x2 typically requires less memory allocation, but it may encounter unreliable reference frame counts in the source's SPS, potentially causing playback failures. Therefore, this configuration should only be implemented after thorough source compatibility testing.

1.6 Disable Decoder FBC Output Mode

FrameBuffer Compression (FBC) is a hardware-based frame buffer compression technology. Configuring FBC decoding output can improve decoding efficiency and reduce DDR bandwidth load. On chips such as RK3576, RK3588, and RK356X, FBC output is enabled by default for source materials with resolutions greater than 1080P.

The Codec2 framework provides user interface configuration support to disable FBC decoding output.

Enabling Methods:

```
mediaFormat.setInteger("vendor.c2-dec-fbc-disable.value", 1);
```

Effective Log Confirmation:

```
c2_info("got disable fbc request");
```

1.7 SurfaceTexture Decoder Configuration Output Actual Width and Height

Hardware decoders operate with high efficiency on aligned buffers, so the decoding output buffers requested by the framework are all aligned, meaning there are stride-related invalid data in the decoding output. For example, for the H264 1920x1080 format, the requested buffer specification is 1920x1088, and before sending it to the display, the bottom 8 pixels of the virtual height need to be trimmed.

MediaCodec application configurations using SurfaceView utilize the Android SurfaceFlinger rendering framework, where crop parameters are internally handled by the framework. In contrast, when configuring SurfaceTexture, rendering is performed via GL textures, requiring application-level cooperation to complete crop operations, otherwise, green borders may appear during video application testing.

To be compatible with early SurfaceTexture video applications, provide user configuration for direct output without stride to avoid green border issues.

> Compared to the previous process, this method manually copies a decoding output without stride within the framework, which may slightly increase the decoding time of the framework.

Enabling Methods:

```
mediaFormat.setInteger("vendor.c2-dec-output-crop-enable", 1);
```

Effective Log Confirmation:

```
c2_info("got request for output crop");
```

2. Extended Parameter Support for Encoder Platform

2.1 Support for Scalable TSVC Encoding Mode

The SVC (Scalable Video Coding) operates by processing video in a layered manner. It allows video streams to be divided into multiple layers, each of which can be decoded independently. Each layer adds specific details such as higher resolution or smoother motion, and they can be combined to provide different level quality videos.

- Network Adaptability: When network connectivity is unstable, only the base layer video content is decoded. Under smooth network conditions, higher layers containing enhanced information are decoded to provide clearer resolutions and other improvements.
- Error Recovery: The lower-layer bitstream can utilize high-layer information for error detection and correction. If errors are detected in the SVC stream, resolution and frame rate can be progressively downgraded to the base layer. SVC achieves a packet loss resilience of up to 20%.

> SVC technology has natural advantages in fields such as video surveillance and video conferencing, and has therefore been widely used by both domestic and international vendors, including Tencent Rooms conferencing, Microsoft Teams conferencing, and others that employ SVC encoding.

The Rockchip platform provides an implementation of Timing Scalable Video Coding (TSVC) and supports configuration of 2-4 layer encoding:

```
mediaFormat->setString("ts-schema", "android.generic.3");
```

You can verify the effectiveness by printing or viewing the generated code stream below:

```
c2_info("setupTemporalLayers: layers %d", layerCount);
```

2.2 QP Range Setting

QP is the quantization parameter for bitstream encoding quality, with a configurable range of 1 ~ 51. Generally, a smaller QP value results in higher video quality but increases the bitrate correspondingly. Conversely, a larger QP value leads to lower video quality but reduces the bitrate accordingly. Therefore, setting an appropriate QP range is critical for balancing video quality and file size.

Refer to the following code to configure the encoder QP range. Configure the I-frame QP range to 10 ~ 40 and the P-frame QP range to 10 ~ 30.

```
mediaFormat->setInt32("video-qp-i-min", 10);
mediaFormat->setInt32("video-qp-i-max", 40);
mediaFormat->setInt32("video-qp-p-min", 10);
mediaFormat->setInt32("video-qp-p-max", 30);
```

The effectiveness can be confirmed by printing or viewing the generated code stream below:

```
c2_info("setupQp: qpInit %d i %d-%d p %d-%d", qpInit, iMin, iMax, pMin, pMax);
```

2.3 Rotation Encoder Support

Enable rotation angle configuration support, supporting 90\180\270 degrees video rotation angles. Refer to the following code to configure the encoder output stream to rotate 90 degrees clockwise:

```
mediaFormat->setInt32("rotation", 90);
```

The effectiveness can be confirmed through the following print output:

```
c2_info("setupPreProcess: rotation degrees %d", degrees);
```

2.4 Scaling Encoding Support

The platform's hardware does not natively support scaling encoding. This functionality relies on the platform's graphics processing hardware (RGA) to perform scaling preprocessing on the encoding input, thus potentially increasing the encoding time slightly.

Refer to the following code, and configure the encoder bitrate to output at the 720p resolution:

```
mediaFormat->setInt32("vendor.c2-enc-input-scalar.width", 1280);
mediaFormat->setInt32("vendor.c2-enc-input-scalar.height", 720);
```

The effectiveness can be confirmed through the following print output:

```
c2_info("setupBaseCodec: coding %s w %d h %d hor %d ver %d",
        toStr_Coding(mCodingType), mSize->width, mSize->height, mHorStride,
mVerStride);
```

2.5 Multi-slice Configuration Support

H264\H265 encoding uses macroblock (short for MB) as the smallest unit. Multiple consecutive macroblocks form a slice, and each slice generates one NALU (Network Abstraction Layer Unit) during encoding. By default, one frame image is encoded into a single slice.

- Due to the concept of partitioning slices, a slice contains the entire or partial data of a particular frame.
- Slice partitioning can be performed either by a fixed number of macroblocks or by byte size, which corresponds to the cumulative byte count of macroblocks.
- Partitioning into slices results in each slice having its own dedicated packet header and other metadata, thus the final encoded bitrate will be increased accordingly. The finer the partitioning, the higher the overall bitrate.

The main scenarios applicable to multi-slice are as follows:

1. In network flow transmission scenarios with MTU size limitations, such as when the maximum packet size for a single transmission is 1500 bytes, the data can be divided into slices, with each slice sized at 1500 bytes.
2. The encoding of multiple slices is independent. For chips with multi-core encoders such as RK3588, multiple slices can be encoded concurrently to enhance the encoding performance.
3. Anti-network loss functionality, after slicing the network resources, implementing error correction mechanisms allows the corresponding processing area to be minimized, thereby reducing resource consumption.

Currently supports partitioning slices based on byte size by default. Refer to the following configuration:

```
mediaFormat->setInt32("vendor.c2-enc-slice-size.value", 1500);
```

The effectiveness can be confirmed through the following print output:

```
c2_info("setupSliceSize: slice-size %d", c2Size->value);
```

2.6 Ultra-large Frame Re-encoding Support

Ultra-large frames impose significant impact on the network, and this configuration is suitable for scenarios requiring smooth bitrate refresh.

Note that the encoding time per frame may become longer with more re-encoding times for ultra-large frames. Please configure according to actual requirements. To configure the encoder's re-encoding times for ultra-large frames to three at most, refer to the following configuration:

```
mediaFormat->setInt32("vendor.c2-enc-reenc-times.value", 1500);
```

The effectiveness can be confirmed through the following print output:

```
c2_info("setupReencTimes: reenc-times %d", reencTime->value);
```

2.7 RK3576\RK3588 Smart Encoding Mode Support(Smart Bitrate Reduction)

Super Encoding Mode is developed based on customer requirements for bandwidth saving\image quality enhancement, etc. Currently, the platform primarily develops three versions:

- 1.0 Version: Based on inter-frame\intra-frame complexity information, ROI (Region of Interest) area proportion, and maximum/minimum bitrate, adaptively adjust quantization parameters to optimize encoding strategies and achieve bitrate savings. The bitrate optimization for large motion, small motion, and static scenes has been improved by +21%, +85%, and +90%, respectively.
- 2.0 Version: Enhanced statistical information and smoother rate control strategy. Further optimized bitrate and clarity in static weak-texture scenarios.
- 3.0 version: Introduced AI neural network intelligent recognition of ROI (Region of Interest) areas to achieve the purpose of image quality enhancement.

> Note that currently only RK3576\RK3588 support the smart encoding mode.

Currently, the MediaCodec extended parameter set supports configuration of Smart Encoding versions 1.0 & 3.0.

The platform provides multiple strategies for bitrate control optimization. The following code configures the image quality priority mode under motion scenarios:

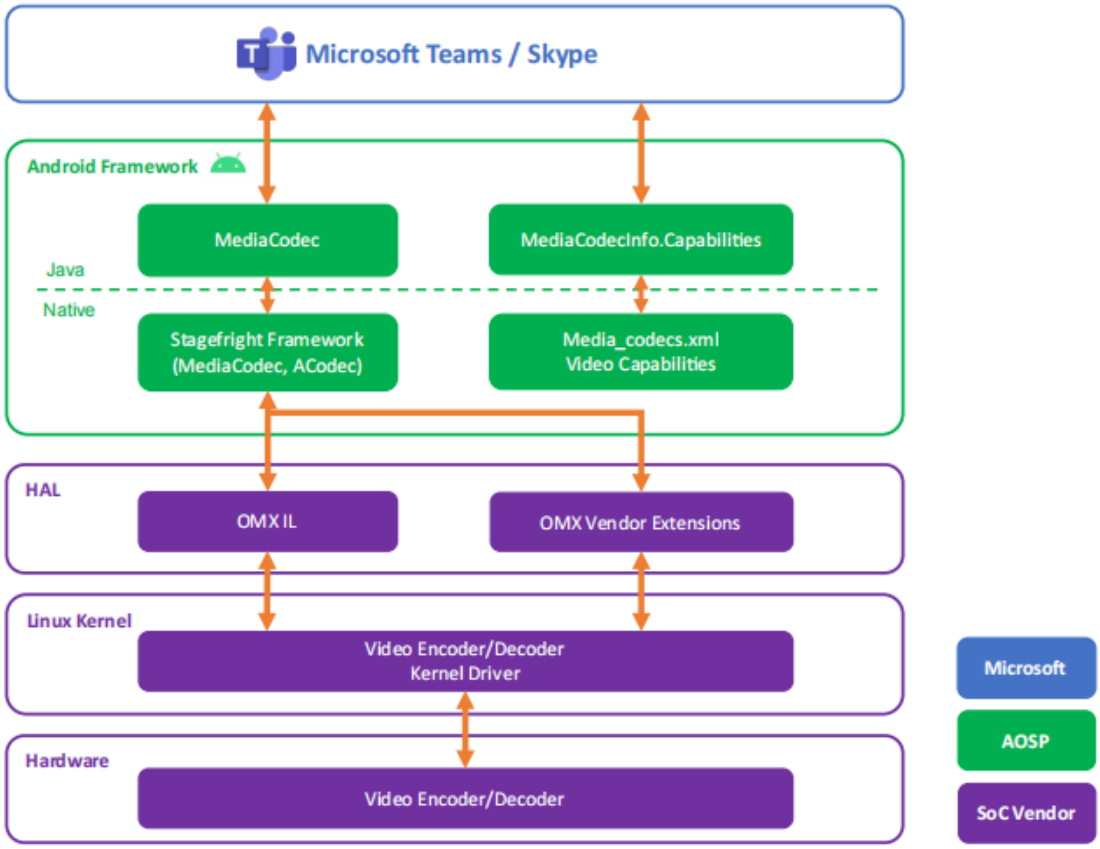
```
mediaFormat->setInt32("vendor.c2-enc-super-encoding-mode.value", 1);

// Set Value:
// 1    ===== Image quality prioritized, aiming for higher PSNR and VMAF values
// 2    ===== Bitrate prioritized, aiming for higher video compression rates
// 3    ===== smart v3.0 mode, ROI adaptive recognition, enhanced image quality
```

The effectiveness can be confirmed through the following print output:

```
c2_info("setupSuperMode: setup super mode %d", superMode);
```

2.8 Microsoft MVCT Protocol Certification Support



MVCT (Microsoft Codec Validation Tool) is the component responsible for codec protocol validation in the Microsoft Teams meeting system. It adapts to and implements the extended protocol interfaces required by Microsoft, supporting functionalities such as TSVC, long/short reference frame configuration, frame-level QP settings, multi-slice partitioning, and dynamic bitrate and resolution adaptation.

Microsoft Teams certification is a rigorous and long process with strict requirements for the performance of hardware devices, including media\camera\audio. Customer-certified devices must individually pass:

- MVCT codec protocol authentication, supports adaptation for the Microsoft MVCT extension protocol, adds encoding prefix NAL information, and has fully passed Microsoft MVCT testing and certification.
- VCT certification and pressure test certification require that under multi-channel encoding/decoding and multi-display pressure testing scenarios, key metrics such as performance\CPU usage\memory occupancy rate remain within normal ranges at all times.
- Device hardware certification, including Audio\Camera\optical and other hardware certifications.

Currently Rockchip platform has successfully adapted to support the MCVT codec protocol from Microsoft on RK3588\RK3399\PX30 chips, enabling relevant devices to pass Teams certification. The RK3588 device, featuring high-performance capabilities, has been included in Microsoft's recommended selection for domestic chip options.

MVCT Test Tool

2.9 SEI Encoding Supplementary Enhanced Information Disable Support

Supplemental Enhancement Information (SEI) is a concept within the bitstream context, providing a method for adding information to the video bitstream. It is one of the features of the H264/H265 video compression standards.

SEI is typically integrated before the video stream SPS/PPS/IDR, adding user-defined information such as encoder parameters\video copyright information\camera parameters, etc. These information are helpful for the decoding process (error detection, error correction), but they are not essential for the decoding process.

Due to the requirements of Google GMS, the RK encoder defaults to inserting a frame of SEI at the initial position of the bitstream. However, it has been observed that some older decoders do not support SEI, leading to decoding failures. Therefore, user extension parameters are provided to allow users to disable the output of SEI information.

```
mediaFormat->setInt32("vendor.c2-enc-disable-sei.value", 1);
```

This can be confirmed effective through the following print:

```
c2_info("disable sei info output");
```

2.10 Encoding ROI Region of Interest Settings

ROI (Region of Interest) is a video encoding technology based on regions of interest, which reduces the quantization parameter values for the regions of interest in the image, thereby allocating more bitrate to improve the encoding quality. For regions that are not of interest, the quantization parameter values are increased, resulting in less bitrate allocation. Under the same bitrate constraint, this approach achieves better subjective visual quality.

The design concept of the ROI video encoding application using MediaCodec is based on identifying ROI regions (such as face areas, central regions, etc.) during video pre-processing, and then passing the ROI region information along with encoding quality adjustment parameters through the MediaCodec->setParameter method.

Parameter Configuration Instructions:

1. Supports up to 4 ROI regions, identified using vendor extension parameter prefixes "roi-region-config" to "roi-region4-config".
2. The key parameter specifies the specific ROI (Region of Interest) area and encoding quantization parameters, with the following meanings:
    - left: The horizontal coordinate value of the top-left corner of the ROI region

- right: The vertical coordinate value of the top-left corner of the ROI region
- width: width of the ROI region
- height: Height of the ROI region
- force-intra: Whether to specify ROI region encoding as I-frames
- qp-mode: Configuration value 0 - qpVal is a relative value / 1 - qpVal is an absolute value
- qp-val: The quantization parameter (QP) value of the macroblock within the Region of Interest (ROI).

3. When qp-mode is set to a relative value, the configured qp value is the sum of the qp generated by bitrate control and the user-defined qp offset.
4. Parameter configuration is set to 'one-time', and each time before queueInputBuffer, the regional configuration needs to be updated.

Configuration Example: Configure two 160x160 ROI (Region of Interest) areas and specify the QP (Quantization Parameter) value as 10:

```
mediaFormat->setInt32("vendor.c2-enc-roi-region-config.left", 0);
mediaFormat->setInt32("vendor.c2-enc-roi-region-config.right", 0);
mediaFormat->setInt32("vendor.c2-enc-roi-region-config.width", 160);
mediaFormat->setInt32("vendor.c2-enc-roi-region-config.height", 160);
mediaFormat->setInt32("vendor.c2-enc-roi-region-config.force-intra", 0);
mediaFormat->setInt32("vendor.c2-enc-roi-region-config.qp-mode", 1);
mediaFormat->setInt32("vendor.c2-enc-roi-region-config.qp-val", 10);

mediaFormat->setInt32("vendor.c2-enc-roi-region2-config.left", 500);
mediaFormat->setInt32("vendor.c2-enc-roi-region2-config.right", 500);
mediaFormat->setInt32("vendor.c2-enc-roi-region2-config.width", 160);
mediaFormat->setInt32("vendor.c2-enc-roi-region2-config.height", 160);
mediaFormat->setInt32("vendor.c2-enc-roi-region2-config.force-intra", 0);
mediaFormat->setInt32("vendor.c2-enc-roi-region2-config.qp-mode", 1);
mediaFormat->setInt32("vendor.c2-enc-roi-region2-config.qp-val", 10);
```

This can be confirmed effective through the following print:

```
c2_info("setup roi done, ctx %p regionCount %d", mRoiCtx, regionCount);
```

2.11 Encoding pre-processing support, Mirror\up-down flipping

Support encoder preprocessing, including mirroring and flipping, as demonstrated in the following code. Enable encoder preprocessing using the extended parameter interface:

```
format->setInt32("vendor.c2-enc-preprocess.mirror", 1)
format->setInt32("vendor.c2-enc-preprocess.flip", 1)
```

This can be confirmed effective through the following print:

```
c2_info("setupPreProcess: mirroring");
c2_info("setupPreProcess: flip");
```